



Material de Apoio

Oficina Online de Construção de Mathlets com Streamlit

Transformando a Análise Real em Aplicações Interativas

Profa. Dra. Greice Keli Silva Lacerda
Profa. Dra. Márcia Maria Fusaro Pinto

Projeto Math<<Fusion>>Tech | DEPEXT

Abril – Junho de 2026

Conteúdo

Apresentação	4
Primeiros Passos Técnicos	5
1 Encontro I: Lógica de Sequências (22/04)	9
1.1 IA como Mentoria: O Uso Ético na Programação	9
1.2 Sequências Reais: Do Infinito ao Discreto	10
1.3 Provocação Matemática	14
1.4 Como Salvar seu Progresso: O Ciclo de Autoria no GitHub	15
1.5 Desafio! Para praticar até o próximo encontro	17
1.6 Reflexão para o Professor-Autor	18
2 Encontro II: Interfaces para Análise de Funções e Continuidade Local (06/05)	19
2.1 O Rigor Analítico: Definição Formal de Continuidade	19
2.2 A Análise através da Interface	19
2.3 Questões para Reflexão: A Dialética entre o Código e a Análise	20
2.4 Desafio Assíncrono! Onde a Intuição Falha	21
3 Encontro III: Somas de Riemann (22/05)	23
3.1 Rigor Formal: A Definição de Riemann	23
3.2 Visualização com Interface: O Refinamento da Partição	23
3.3 Questões Reflexivas: A Convergência em Foco	24
3.4 Desafio Assíncrono: O Erro e a Precisão	25
4 Encontro IV: Deploy e Avaliação (03/06)	26
4.1 Configuração de Dependências	26
4.2 Justificativa Teórico-Técnica das Bibliotecas	26
4.3 Passo a Passo para Publicação no Streamlit Cloud	27
4.4 O App Publicado	31
4.5 Questões Reflexivas: O Professor como Designer	32
Considerações Finais	33
Referências Bibliográficas	34
Apêndice A: Guia de Configuração do Ambiente de Desenvolvimento	35
Apêndice B: Código-Fonte do Visualizador de Vizinhanças	36
Apêndice C: Protótipo de Mathlet para Convergência de Sequências	37

Apêndice D: Estrutura de Integração (Somas de Riemann)	38
Apêndice E: Código-Fonte para Análise de Continuidade Local ($\epsilon - \delta$)	39
Apêndice F: Código-Fonte para Somas de Riemann e Cálculo de Erro	42
Apêndice G: Instrumentos de Coleta de Dados (Questionários de Encontro)	44

Apresentação

Bem-vindo(a) à Oficina **Math⟨⟨Fusion⟩⟩Tech!**

Esta jornada de formação foi concebida como parte de uma pesquisa de doutorado que investiga como a **autoria digital** pode ressignificar o aprendizado da Análise Real. Frequentemente percebida como uma disciplina de alta abstração e rigor estático, a Análise ganha, nesta oficina, uma nova dimensão: a dimensão da interatividade e do movimento.

Por que programar em Análise Real?

A proposta desta intervenção, certificada pela **DEPEXT**, fundamenta-se na ideia de que *programar é uma forma de explicar para o computador um conceito matemático*. Ao transcrever teoremas, sequências e somas de Riemann para a linguagem Python, você deixará de ser apenas um usuário de tecnologias prontas para se tornar um **Designer de Mathlets**.

Nesta oficina, utilizaremos:

- **Python + Streamlit:** Ferramentas de ponta para criar aplicativos web científicos com baixíssima curva de aprendizado técnico.
- **GitHub Codespaces:** Um ambiente de desenvolvimento 100% em nuvem, eliminando barreiras de instalação.
- **IA como Mentoria:** O uso ético e estratégico de Inteligência Artificial para auxiliar na sintaxe do código, permitindo que seu foco permaneça na lógica matemática.

O Percurso de Aprendizagem

Nosso cronograma está organizado para que, ao final dos quatro encontros, você tenha construído e publicado seu próprio repositório de ferramentas didáticas interativas. Esperamos que este material de apoio seja um guia útil para transformar o “desafio” da Análise em um estímulo criativo e pedagógico.

“A tecnologia não substitui o rigor, ela o torna visível.”

Profa. Dra. Greice Keli Silva Lacerda
Profa. Dra. Márcia Maria Fusaro Pinto
Projeto Math⟨⟨Fusion⟩⟩Tech / DEPEXT

Preparação Obrigatória: Primeiros Passos Técnicos

Olá pessoal! Para que possamos iniciar nossa oficina **Math<<Fusion>>Tech** com foco total na Matemática, é fundamental que você realize as configurações técnicas abaixo antes do nosso primeiro encontro.

1. Entre no Grupo da Oficina

Utilizaremos o WhatsApp para comunicações rápidas, avisos e suporte técnico imediato.

- **Link do Grupo:** [Math<<Fusion>>Tech](#)

2. Crie sua conta no GitHub

O GitHub será nossa “nuvem” de desenvolvimento.

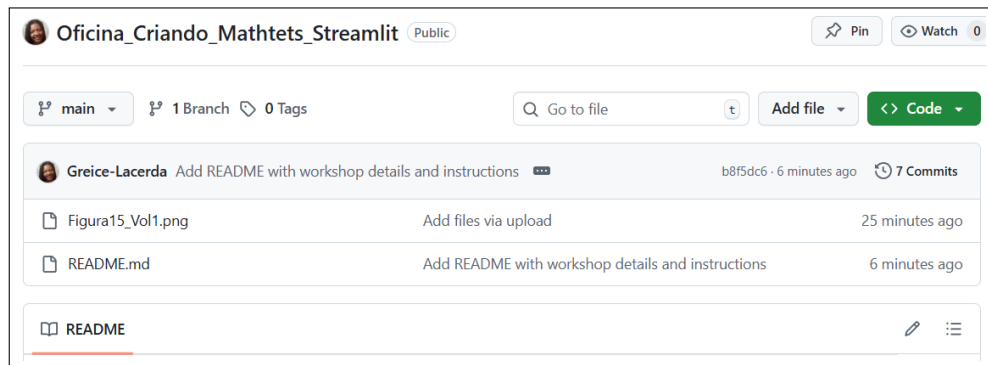


1. Acesse github.com/join.
2. Siga as instruções para criar e verificar sua conta.
3. *Dica:* Se você tiver um e-mail institucional, poderá solicitar o *Student Developer Pack* futuramente.

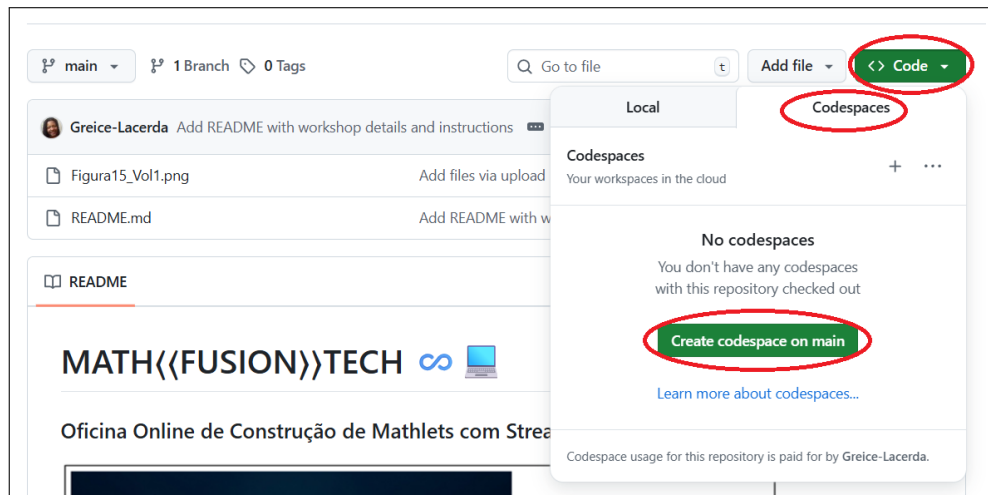
3. Acesse o Repositório e o Codespace

O Codespace é um computador na nuvem pronto para programar, sem precisar instalar nada no seu PC físico.

1. Acesse o repositório da oficina: [Oficina_Criando_Mathtets_Streamlit](#).



2. Clique no botão verde <> Code.
3. Selecione a aba **Codespaces** e clique em **Create codespace on main**.



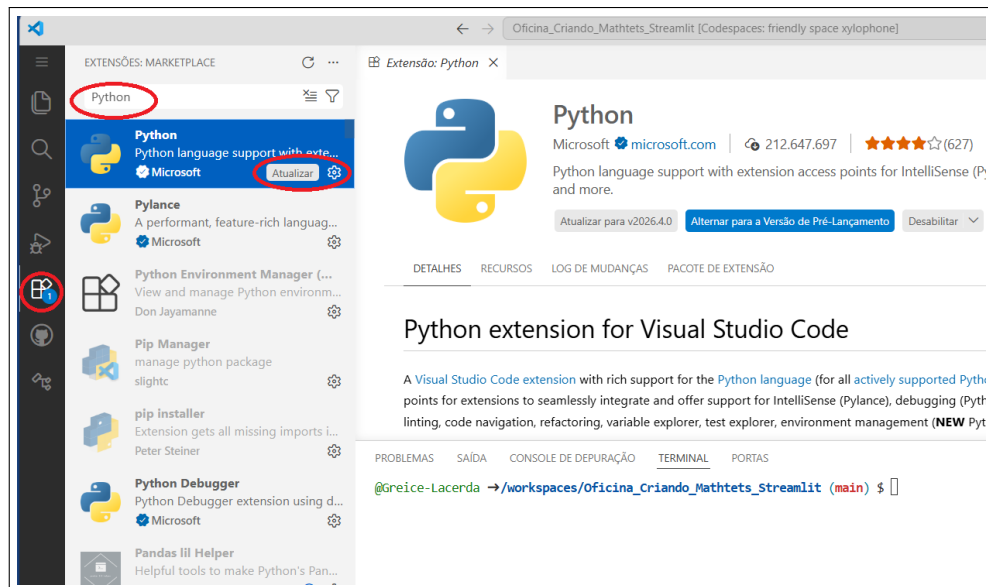
4. Aguarde alguns segundos até que o editor (VS Code Web) abra no seu navegador.



4. Instalando as Ferramentas de Autoria

Dentro do ambiente Codespace que você acabou de abrir, precisamos garantir que o Python e o Streamlit funcionem corretamente:

- **Python Extension:** No menu lateral esquerdo, clique no ícone de quadrados (Extensions), digite "Python" e clique em **Install** (caso ainda não esteja instalado).

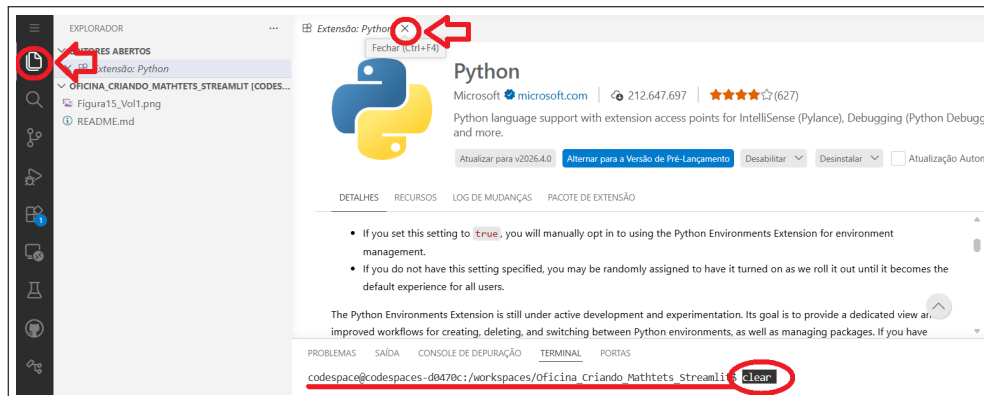


- **Instalando o Streamlit:** No terminal (parte inferior da tela), digite o seguinte comando e aperte *Enter*:

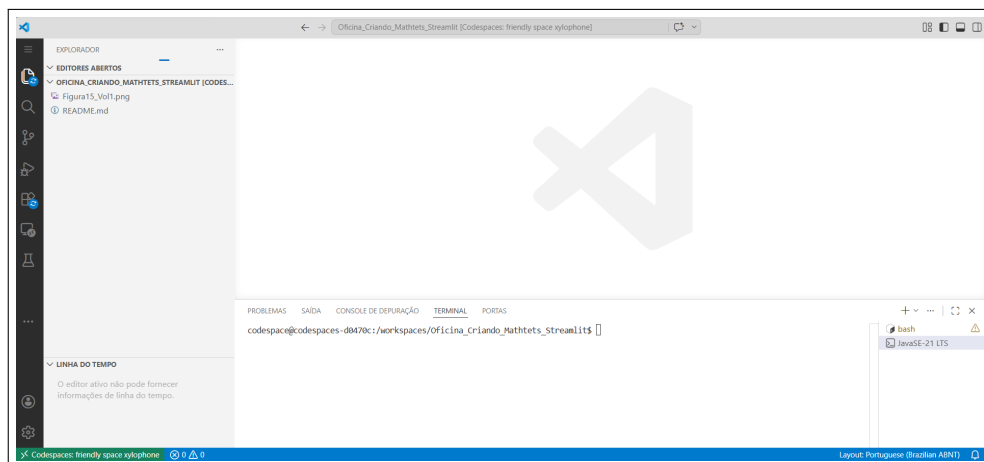
```
1 pip install streamlit matplotlib numpy
```



- Vamos organizar seu espaço de trabalho? Observe as marcações em vermelho na imagem a seguir.



1. Clique no botão Explorador, no canto superior esquerdo, para acessar os arquivos que editaremos.
 2. Clique no "X", acima do nome Python, na tela central para fechar a janela de instalação.
 3. Digite "clear" no terminal para limpar o processo de instalação e ficar como o terminal menos poluído.
- Agora o seu espaço de desenvolvimento está equipado e pronto para a criação.



"A preparação técnica é o primeiro passo para a liberdade criativa na Análise Real."

Dúvidas? Você pode enviar uma mensagem no grupo da oficina!

Nos vemos dia 22/04.

1 Encontro I: Lógica de Sequências (22/04)

Objetivo

Desmistificar a programação e traduzir o conceito de convergência para o pensamento computacional.

1.1 IA como Mentoria: O Uso Ético na Programação

Nesta oficina, utilizaremos assistentes de Inteligência Artificial (como ChatGPT ou GitHub Copilot) de forma estratégica. Programar em Análise Real exige precisão; portanto, a IA não deve substituir o seu raciocínio matemático, mas sim auxiliá-lo na sintaxe e na depuração (*debugging*) de erros.

Diretrizes para o Uso Ético

1. **Não copie sem entender:** Cada linha gerada pela IA deve ser explicada por você. Se não sabe o que um comando faz, pergunte à IA: *"O que a função `np.linspace` faz neste contexto?"*
2. **Validação Matemática:** A IA pode cometer erros conceituais (alucinações). Sempre verifique se o resultado visual do gráfico condiz com as definições da Análise Real.
3. **Prompt Engineering Pedagógico:** Em vez de pedir "faça o código", peça "explique o erro no meu código" ou "sugira uma forma de otimizar esta visualização".

Exemplo de Prompt Pedagógico

Para garantir que a IA atue como sua mentora, utilize a estrutura abaixo. Note que não pedimos apenas o resultado, mas a **explicitação do pensamento matemático**.

Prompt para Praticar

"Explique-me o conceito de sequência de números reais em Análise Real. Em seguida, crie um código em Streamlit que permita ao usuário escolher uma sequência numérica entre cinco já pré-selecionadas e controlar o valor de n em um gráfico, visualizando seus pontos discretos. Comente cada linha explicando a relação entre o código e a definição formal."

1.2 Sequências Reais: Do Infinito ao Discreto

A transição do pensamento contínuo para o pensamento algorítmico exige uma compreensão profunda sobre o domínio das funções. Nesta seção, exploraremos como representar sequências numéricas de forma que o rigor da Análise Real seja preservado na tela do computador.

Definição Formal: Sequência de Números Reais

Uma **sequência de números reais** é uma função $f : \mathbb{N} \rightarrow \mathbb{R}$, que associa a cada número natural n um número real x_n (denominado termo da sequência).

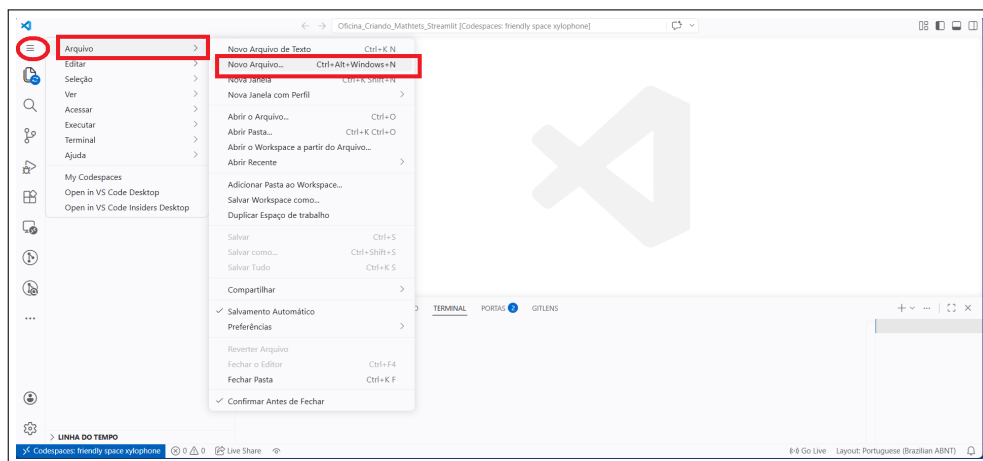
O Olhar da Análise

Diferente das funções reais de variável real $f(x)$, onde o domínio é contínuo, as sequências possuem um **domínio discreto** ($\mathbb{N} = \{1, 2, 3, \dots\}$). Na computação, isso significa que não devemos “ligar os pontos” com linhas, pois não existem valores definidos entre os números inteiros n e $n + 1$.

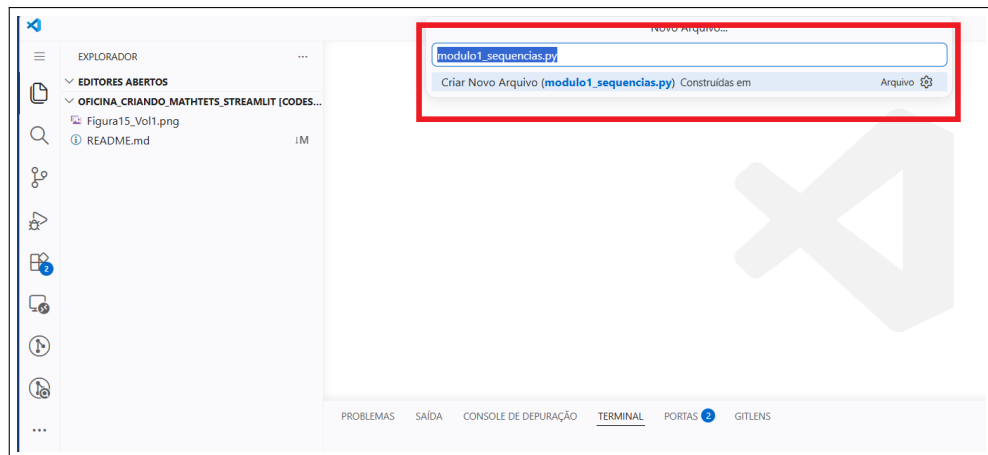
Implementação: Explorador Interativo de Sequências

Vamos criar nosso primeiro arquivo de código para visualizar esse conceito?

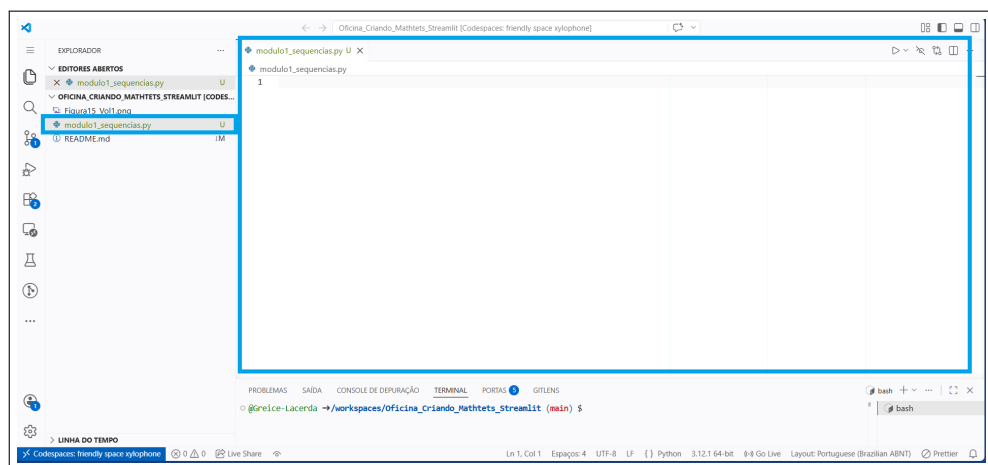
1. Acesse o seu Codespace no repositório do GitHub: Use o mesmo link dos primeiros passos; porém, agora não aparecerá mais o botão "**Create codespace on main**". No lugar, estará o link de acesso a seu espaço de código. Caso não esteja, crie-o novamente.
2. Agora, vamos criar nosso primeiro arquivo. No menu lateral, acesse “Arquivo” > “Novo Arquivo”. Veja a imagem:



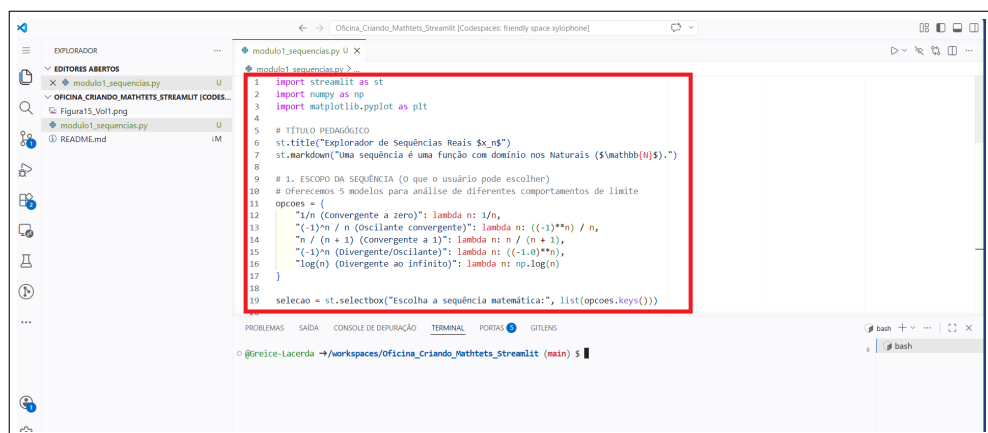
3. Na janela central, digite o nome o arquivo `modulo1_sequencias_Iniciais_Nome.py` e Clique em “Criar”:



4. O seu arquivo será adicionado no “Explorador” e o editor abrirá um espaço para inserir seu código:



5. Copie e cole o código gerado pela sua IA no editor.



O código abaixo é um exemplo que pode ser gerado pela IA. Nesse exemplo, a lógica construída permite ao licenciando escolher entre diferentes tipos de comportamentos (convergência, divergência e oscilação) e controlar o avanço do índice n .

```

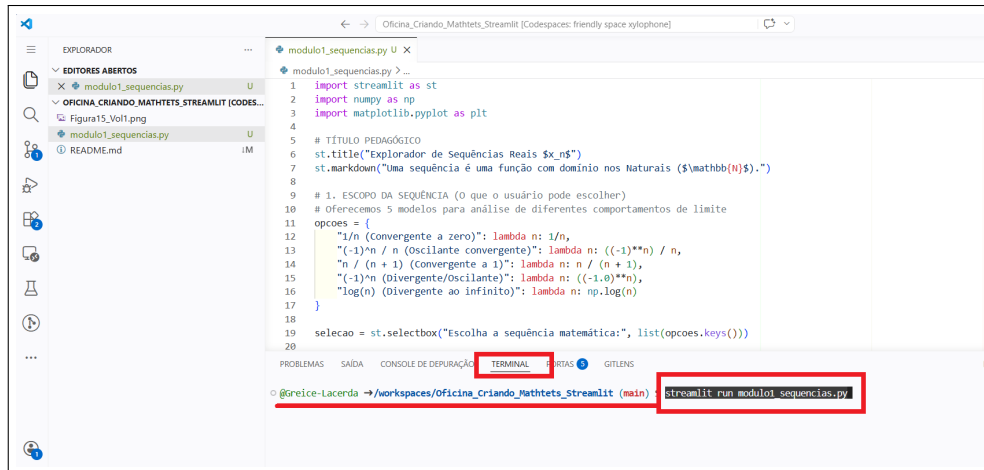
1  import streamlit as st
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  st.title("Explorador de Sequências Reais  $x_n$ ")
6
7  # 1. Definição das Sequências (Dicionário de Funções)
8  # Cada chave representa uma regra de forma matemática
9  sequencias = {
10     "1/n (Convergente a 0)": lambda n: 1/n,
11     "(-1)^n / n (Oscilante convergente)": lambda n: ((-1)**n) / n,
12     "n / (n + 1) (Convergente a 1)": lambda n: n / (n + 1),
13     "(-1)^n (Divergente Oscilante)": lambda n: ((-1.0)**n),
14     "log(n) (Divergente ao Infinito)": lambda n: np.log(n)
15 }
16
17 selecao = st.selectbox("Selecione a sequência:", list(sequencias.
18 keys()))
19
20 # 2. Controle do índice n (0 'n' da definição formal)
21 n_max = st.slider("Visualizar até o termo n =", 5, 100, 20)
22
23 # 3. Tradução do Domínio Discreto
24 # np.arange garante que n seja apenas inteiros (1, 2, 3...)
25 n_valores = np.arange(1, n_max + 1)
26 x_n = sequencias[selecao](n_valores)
27
28 # 4. Construção do Gráfico de Dispersão
29 fig, ax = plt.subplots()
30 # O comando 'scatter' fundamental: ele desenha pontos isolados,
31 # respeitando a natureza discreta do domínio dos Naturais.
32 ax.scatter(n_valores, x_n, color='blue', label=f"Termos  $x_n$ ")
33
34 # Linha de referência no zero para observar a convergência
35 ax.axhline(0, color='black', linestyle='--', alpha=0.5)
36
37 ax.set_xlabel("Índice (n  $\in \mathbb{N}$ )")
38 ax.set_ylabel("Valor do Termo ( $x_n$ )")
39 ax.grid(True, linestyle=':', alpha=0.6)
40

```

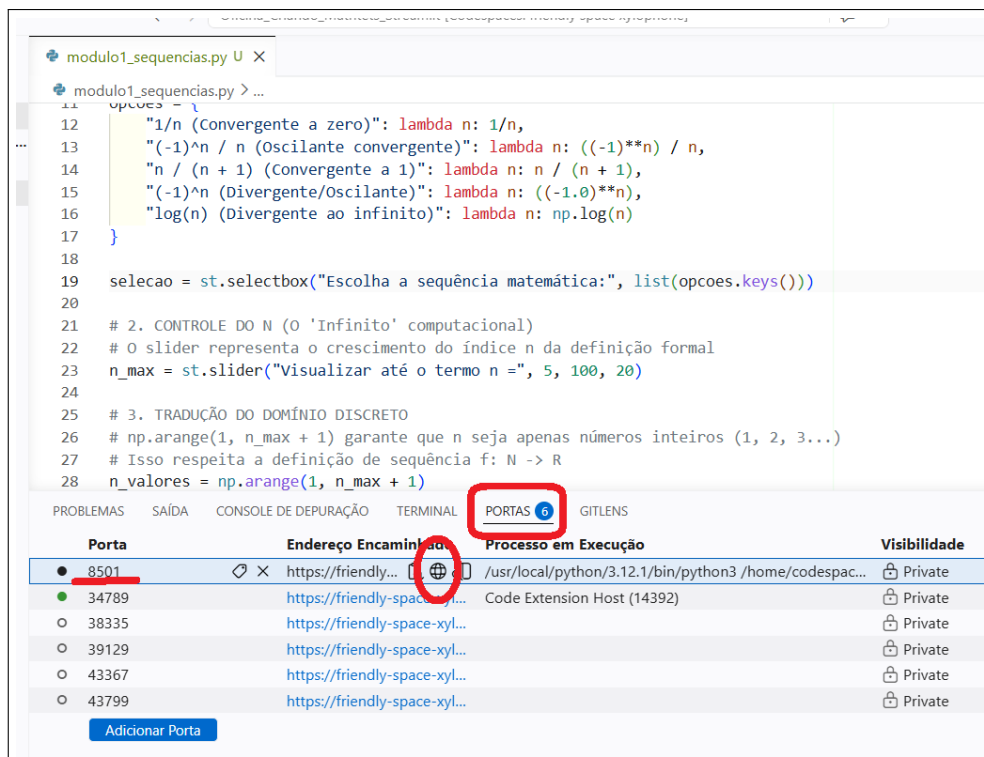
41 `st.pyplot(fig)`

Código 1: Explorador de Sequências com Domínio Discreto

6. Para explorar interativamente as sequências digite no “Termminal” os seguintes comando: “`streamlit run modulo1_sequencias.py` e dê “Enter”:



7. Como estamos trabalhando como o VS Code no GitHub, a visualização não abrirá automaticamente (como ocorre no VS Code Desktop). Então precisamos acessar a porta “8501” e clicar no ícone do globo. Veja a imagem:



A página de exploração abrirá em uma nova janela e você poderá explorar o Mathlet que acabou de construir.

Explorador de Sequências Reais x_n

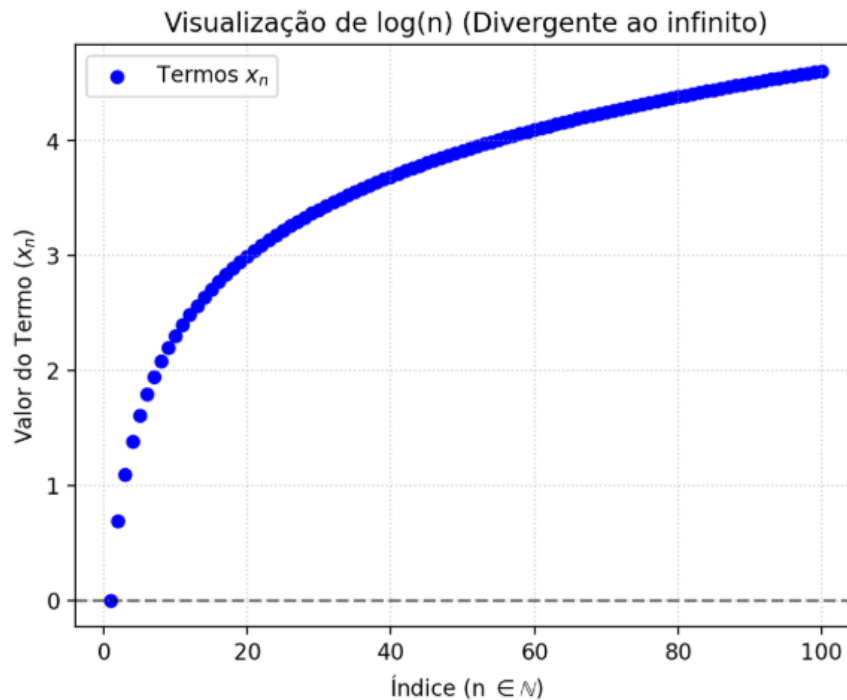
Uma sequência é uma função com domínio nos Naturais ($\mathbb{N}$).

Escolha a sequência matemática:

log(n) (Divergente ao infinito)

Visualizar até o termo n =

100



Último termo calculado: $x_{100} = 4.6052$

1.3 Provocação Matemática

- Ao deslizar o *slider* para valores cada vez maiores de n , o que acontece com a distância entre os pontos e o eixo horizontal? Como essa visualização se conecta com a definição formal de Limite de uma Sequência:

Dizemos que uma sequência de números reais (a_n) converge para um limite $L \in \mathbb{R}$, e escrevemos $\lim_{n \rightarrow \infty} a_n = L$, se for satisfeita a seguinte condição:

Definição

$$\forall \epsilon > 0, \exists n_0 \in \mathbb{N} \text{ tal que } \forall n > n_0 \implies |a_n - L| < \epsilon$$

- Por que usamos `ax.scatter` (pontos) em vez de `ax.plot` (linha)? Como isso se conecta com a definição de função de domínio discreto?
- Se aumentarmos o `n_max` para 1000, o computador consegue nos mostrar o limite? Discuta a diferença entre a “tendência” visual e a prova formal $\forall \epsilon > 0$.
- Observe a sequência $(-1)^n$. Por que ela desafia a intuição visual de aproximação em comparação com a sequência $1/n$?

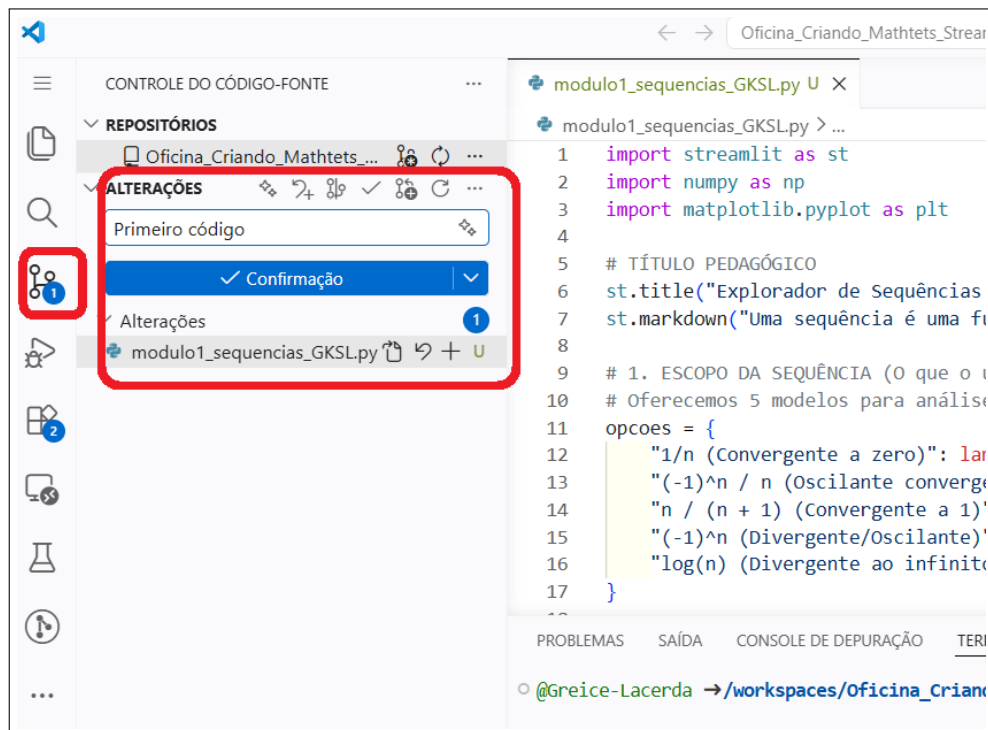
1.4 Como Salvar seu Progresso: O Ciclo de Autoria no GitHub

Diferente de um editor de texto comum, no **GitHub Codespaces** o salvamento ocorre em duas etapas. É fundamental que você realize o “Commit” para que suas criações fiquem registradas permanentemente no seu repositório.

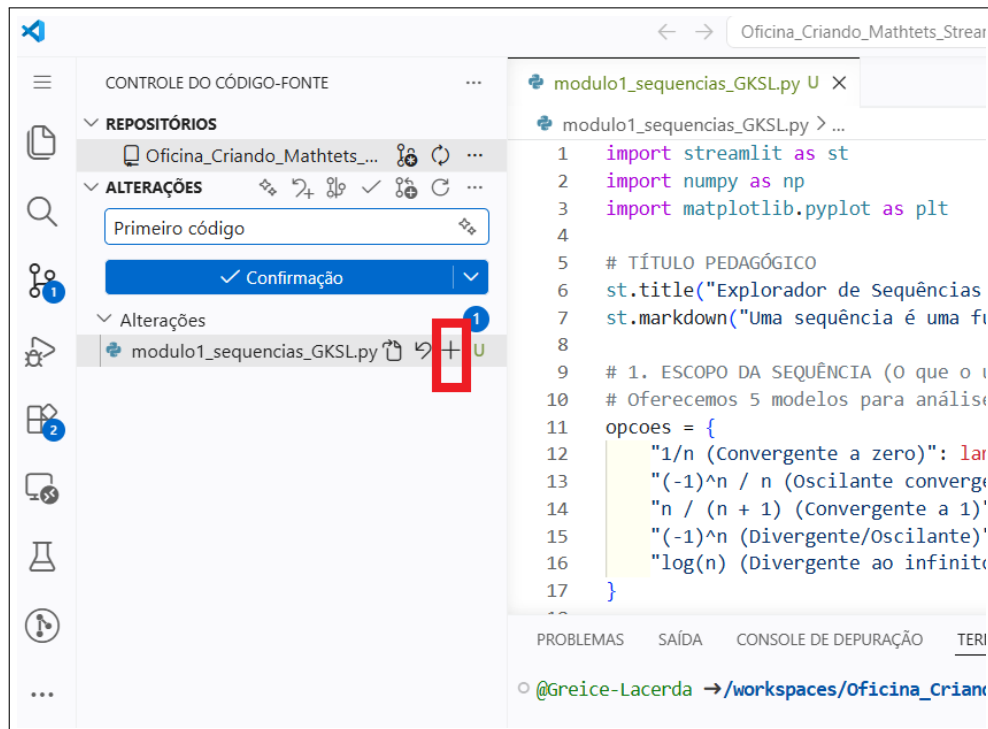
Passo a Passo para o Salvamento (Commit & Push)

Siga estes passos ao final de cada atividade ou antes de fechar o navegador:

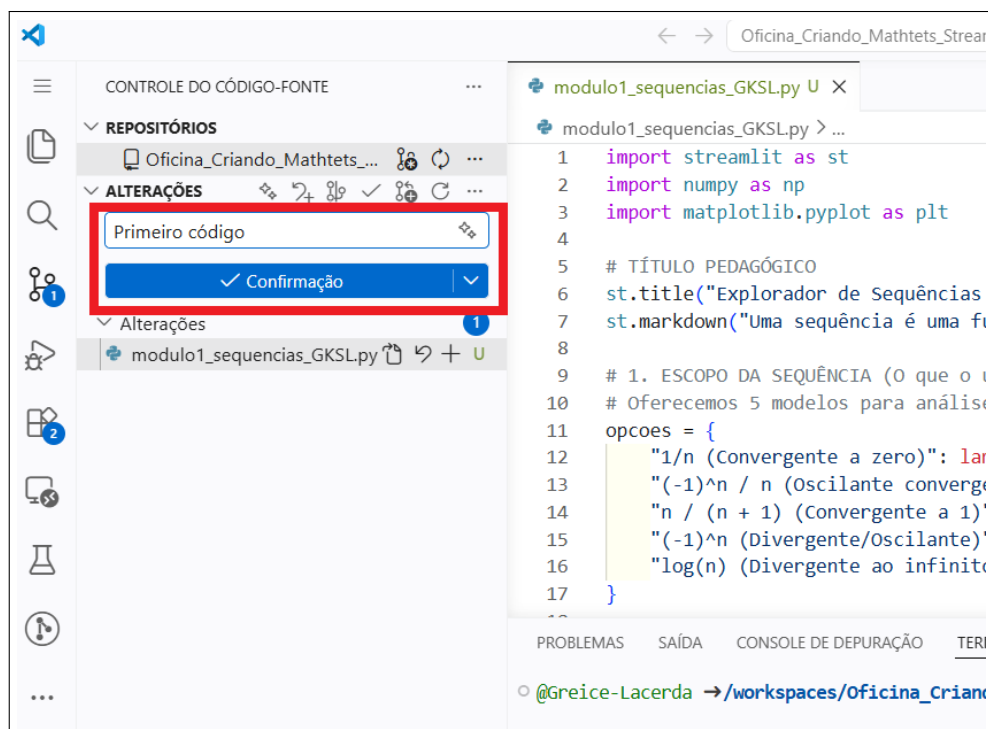
1. **Source Control:** No menu lateral esquerdo do Codespaces, clique no ícone de **Controle de Versão** (ícone que parece um grafo com três pontos conectados).



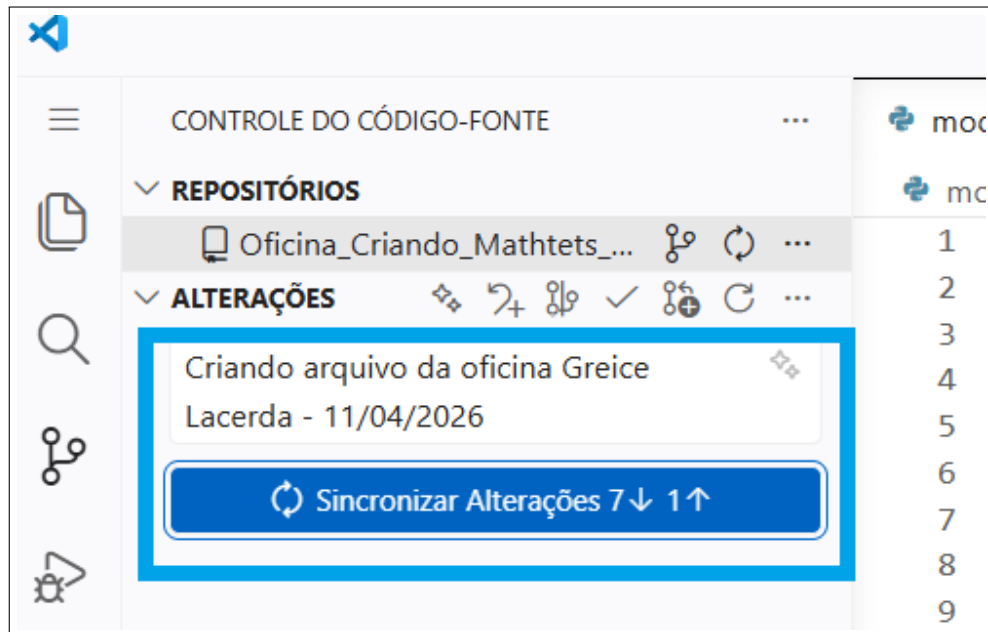
2. **Preparar (Stage):** Você verá seus arquivos na lista "Changes". Clique no ícone de + (mais) ao lado do nome do seu arquivo (ex: modulo1_sequencias.py). Ele subirá para a seção "Staged Changes".



3. **Mensagem e Confirmação:** Na caixa de texto acima, escreva uma breve descrição do que você fez. Exemplo: *"Conclusão da atividade de Sequências - [Seu Nome]"*. E para **Confirmar (Commit)**: Clique no botão azul **Commit**.



4. **Sincronizar (Push):** Clique no botão **Sync Changes** (ou *Publish Branch*) para enviar o código do ambiente virtual para o seu perfil no GitHub.



Atenção!

Se você apenas fechar a aba do navegador sem realizar o **Commit**, suas alterações podem ser perdidas quando o Codespace entrar em inatividade. O Commit é a sua assinatura digital como autor do Mathlet.

1.5 Desafio! Para praticar até o próximo encontro

Este desafio foi desenhado para consolidar sua autonomia no ambiente *GitHub Codespaces*. O objetivo é que você atue como **professor-autor**, transcrevendo o rigor da Análise Real para um *Mathlet* interativo.

Atividade: Visualizador de Vizinhanças

O conceito de proximidade em $\mathbb{R}$ é o pilar para as definições de limite e continuidade. Sua tarefa é criar um aplicativo que materialize a ideia de vizinhança.

Recordação: Definição de Vizinhança

Seja $a \in \mathbb{R}$ e $\delta > 0$. A **vizinhança de raio δ do ponto a** é o conjunto:

$$V_{\delta}(a) = \{x \in \mathbb{R} : |x - a| < \delta\}$$

Mão na Massa: Implementação

1. No seu Codespace, crie um novo arquivo chamado:
`modulo1-vizinhanca-[suas-iniciais].py`
2. Use a IA para explicar o conceito, ofereça um exemplo de código e a explicação detalhada de como cada linha de código se relaciona com o conceito. O apêndice B apresenta um exemplo de código.
3. Implemente o código e salve-o no seu repositório.

1.6 Reflexão para o Professor-Autor

Após rodar o comando `streamlit run [NOME DO ARQUIVO]` e visualizar seu gráfico, reflita para a discussão no próximo encontro:

- **Discretização:** O computador desenha uma mancha verde contínua. Contudo, o código utiliza `np.linspace` ou intervalos de pixels. Como você explicaria a um aluno a diferença entre a "densidade" dos Reais e a representação na tela?
- **O Slider:** O movimento do *slider* ajuda a entender que a definição deve valer para **qualquer** $\delta > 0$?

Finalizando a Atividade

Não esqueça de realizar o **Commit** e o **Push** das suas alterações para o GitHub. Seu processo de criação é parte fundamental desta pesquisa!

2 Encontro II: Interfaces para Análise de Funções e Continuidade Local (06/05)

Objetivos

Neste encontro, o objetivo é a investigação da estabilidade das funções reais, transformando abstrações topológicas em grandezas manipuláveis e verificáveis computacionalmente.

2.1 O Rigor Analítico: Definição Formal de Continuidade

A intuição de que uma função é contínua quando seu gráfico não apresenta "pulos" é o ponto de partida, mas a Análise Real exige uma caracterização local baseada em vizinhanças. No código que desenvolvemos, traduzimos exatamente a definição de Cauchy.

Definição de Continuidade (Local)

Seja $f : X \rightarrow \mathbb{R}$ uma função definida em um subconjunto $X \subset \mathbb{R}$ e $a \in X$ um ponto de acumulação de X . Dizemos que f é **contínua no ponto** a se:

$$\forall \epsilon > 0, \exists \delta > 0 \text{ tal que } \forall x \in X, |x - a| < \delta \implies |f(x) - f(a)| < \epsilon$$

2.2 A Análise através da Interface

Para o analista real, uma função não é apenas um traçado, mas uma relação de proximidade entre espaços. Utilizaremos o *Streamlit* para investigar se, ao aproximarmos valores no domínio (x), a imagem ($f(x)$) responde de forma controlada.

Atividade Prática: Visualizando a Continuidade

Vamos implementar um *Mathlet* que investiga a continuidade de $f(x) = x^2$. O objetivo é permitir que o licenciando tente encontrar um δ que satisfaça a condição de continuidade para um ϵ dado. Use o *prompt* a seguir para solicitar o código e sua explicação linha por linha:

Prompt para Praticar

“Atue como um especialista em Análise Real e programação Python. Preciso criar um *Mathlet* usando *Streamlit* para ensinar a definição de continuidade $\epsilon - \delta$. O código deve permitir que o usuário escolha entre uma função contínua (como $f(x) = x^2$) e uma descontínua (como a função degrau). No gráfico, destaque as vizinhanças $V_\epsilon(f(a))$ e $V_\delta(a)$ usando faixas coloridas. Além do código, explique detalhadamente como a desigualdade $|f(x) - f(a)| < \epsilon$ é verificada dentro do loop ou da lógica do programa.”

Conectando a Definição à Interface

Para interiorizar essa definição, propomos a seguinte correlação entre os símbolos e os elementos do *Mathlet*, cujo exemplo de código encontra-se no Apêndice E:

- $\forall \epsilon > 0$: É o desafio lançado pelo usuário através do *slider* de Epsilon. Representa a "faixa de erro" admissível no eixo das ordenadas (y).
- $\exists \delta > 0$: É a busca que o aluno realiza ao manipular o *slider* de Delta. Representa a "margem de segurança" no domínio (x).
- $|x - a| < \delta$: Graficamente, é o intervalo $(a - \delta, a + \delta)$ destacado pela cor verde no aplicativo.
- $|f(x) - f(a)| < \epsilon$: Graficamente, é a condição de que toda a parte do gráfico contida na faixa verde deve estar, obrigatoriamente, dentro da faixa azul.

Reflexão Analítica

Se, por menor que seja o ϵ , você sempre conseguir encontrar um δ que "prenda" o gráfico dentro da faixa azul, então a função é contínua em a . Caso contrário, se existir um ϵ para o qual nenhum δ (por menor que seja) consiga manter as imagens dentro da faixa, temos uma **descontinuidade**.

2.3 Questões para Reflexão: A Dialética entre o Código e a Análise

Após a exploração do *Mathlet* de continuidade, reflita sobre as questões abaixo:

1. **A Dependência $\delta(\epsilon)$** : Ao manipular os *sliders*, o que acontece com a “margem de erro” no domínio (δ) quando você exige uma precisão maior na imagem

(um ϵ menor)? Como essa experiência sensorial se conecta com a ordem dos quantificadores na definição $\forall \epsilon > 0, \exists \delta > 0$?

2. **O Ponto de Descontinuidade:** Selecione a “Função de Salto (Degrau)”. Tente encontrar um δ para um $\epsilon = 0.1$ no ponto de salto. Por que o computador nunca exibe a mensagem de "Sucesso" nesse caso, independentemente do quão pequeno seja o seu δ ? O que isso nos diz sobre a natureza da descontinuidade?
3. **Discretização vs. Continuidade:** No código, utilizamos `np.linspace` para criar o gráfico. Se tivéssemos apenas 10 pontos entre $a-1$ e $a+1$, nossa verificação de continuidade seria confiável? Como a natureza discreta do computador (pixels e pontos) pode "enganar" a percepção de uma propriedade que é definida para a densidade dos números reais?
4. **O Papel do Programador-Autor:** Ao escrever o bloco `if/else` que valida a condição $|f(x) - f(a)| < \epsilon$, você está, de certa forma, “ensinando” o computador a entender um teorema. Como esse processo de “explicar a matemática para a máquina” alterou a sua própria compreensão da definição formal?
5. **Limitações da Visualização:** Existe algum cenário onde o gráfico parece mostrar que a função é contínua, mas a lógica do código (ou a teoria) diz que não é? (Pense em funções que oscilam infinitamente perto de zero, como $f(x) = \sin(1/x)$).

2.4 Desafio Assíncrono! Onde a Intuição Falha

Para o nosso próximo encontro, você deve atuar como um “detetive analítico”. Até agora, trabalhamos com funções cujo gráfico é “comportado”. O desafio agora é enfrentar a patologia das funções reais.

Atividade: A Oscilação Infinita

Crie um novo arquivo chamado `modulo2-desafio-[suas-iniciais].py` e tente implementar a função:

$$f(x) = \sin\left(\frac{1}{x}\right), \text{ para } x \neq 0$$

Instruções do Desafio

1. Implemente a função usando `np.sin(1/x)`.
2. No seu *Mathlet*, tente analisar a continuidade no ponto $a = 0$.
3. **Experimentação:** O que acontece com o gráfico quando você aproxima o *slider* de a do valor 0? Você consegue encontrar um δ estável para um $\epsilon = 0.5$?
4. **A Questão de Autoria:** Como você configuraria o `np.linspace` para que o gráfico não “engane” o usuário na vizinhança de zero? (Dica: pense no número de pontos da amostragem e use a IA como mentoria).

*Lembre-se de realizar o **git push** com sua tentativa. Não buscamos o código perfeito, mas o registro da sua investigação sobre os limites da representação computacional!*

3 Encontro III: Somas de Riemann (22/05)

Objetivos

Neste encontro, exploraremos a formalização da integral através das Somas de Riemann. O objetivo é compreender como a partição de um intervalo $[a, b]$ e o refinamento dessa partição (fazendo $n \rightarrow \infty$) conduzem ao conceito de integral definida, integrando o rigor da Análise com a visualização dinâmica.

3.1 Rigor Formal: A Definição de Riemann

Na Análise Real, a integrabilidade de uma função está ligada à capacidade de aproximarmos a área sob a curva por retângulos, independentemente de como escolhemos os pontos em cada subintervalo.

Definição Analítica

Seja $f : [a, b] \rightarrow \mathbb{R}$ uma função limitada. Consideremos uma partição $P = \{x_0, x_1, \dots, x_n\}$ do intervalo $[a, b]$. A **Soma de Riemann** de f em relação a P e a um conjunto de pontos intermediários $c_i \in [x_{i-1}, x_i]$ é dada por:

$$S(f, P) = \sum_{i=1}^n f(c_i) \Delta x_i$$

Dizemos que f é integrável segundo Riemann se $\lim_{|P| \rightarrow 0} S(f, P)$ existir e for igual a L . Esse valor L é a integral $\int_a^b f(x) dx$.

3.2 Visualização com Interface: O Refinamento da Partição

Utilizaremos o *Streamlit* para transformar o parâmetro n (número de subintervalos) em uma ação direta. O você verá? À medida que n cresce, a “sobra” dos retângulos diminui.

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 st.title("Somas de Riemann e a Integral Definida")
6
7 # 1. Entrada de Parâmetros
8 a = st.sidebar.number_input("Início do intervalo (a):", value=0.0)
9 b = st.sidebar.number_input("Fim do intervalo (b):", value=2.0)
```

```

10 n = st.sidebar.slider("Número de retângulos (n):", 1, 100, 5)
11
12 # 2. Definição da Função (Exemplo: f(x) = x^2)
13 def f(x):
14     return x**2
15
16 # 3. Cálculo da Soma de Riemann (Pela Esquerda)
17 dx = (b - a) / n
18 x_particao = np.linspace(a, b - dx, n)
19 soma = sum(f(x_particao) * dx)
20
21 # 4. Construção do Gráfico
22 x_curva = np.linspace(a, b, 100)
23 fig, ax = plt.subplots()
24 ax.plot(x_curva, f(x_curva), color='red', label="f(x) = x^2")
25
26 # Desenho dos retângulos
27 for i in range(n):
28     xi = a + i*dx
29     ax.add_patch(plt.Rectangle((xi, 0), dx, f(xi),
30                               edgecolor='blue', facecolor='blue', alpha=0.3))
31
32 ax.set_title(f"Soma de Riemann (n={n}): {soma:.4f}")
33 ax.legend()
34 st.pyplot(fig)
35
36 # 5. Conexão com a Integral Exata
37 st.write(f"Valor aproximado da área : {soma:.4f}")
38 st.latex(r"\int_{a}^{b} f(x) dx \approx \sum_{i=1}^{n} f(x_i) \Delta x")

```

Código 2: Visualizador de Somas de Riemann

3.3 Questões Reflexivas: A Convergência em Foco

1. **O Erro de Aproximação:** O que acontece com a diferença entre a soma dos retângulos e o valor real da integral quando você dobra o valor de n ? É possível chegar ao valor exato apenas aumentando n ?
2. **A Escolha de c_i :** No código, usamos a extremidade esquerda do intervalo ($a + i \cdot dx$). Como o gráfico mudaria se usássemos a extremidade direita ou o ponto médio? A integral mudaria no limite $n \rightarrow \infty$?
3. **Integrabilidade:** Se utilizássemos uma função altamente descontínua (como a de Dirichlet), os retângulos fariam sentido? Como o código reagiria?

3.4 Desafio Assíncrono: O Erro e a Precisão

Desafio para o Encontro IV

Tarefa: Modifique o seu código (salve como `modulo3-riemann-[iniciais].py`) para calcular e exibir a **diferença absoluta** (o erro) entre a Soma de Riemann e o valor exato da integral de x^2 no intervalo escolhido (Dica: a integral de x^2 é $x^3/3$).

Pergunta: Qual é o menor valor de n necessário para que o erro seja menor que 0.01? Use o seu App para descobrir! Um exemplo do código pode se visualizado no Apêndice F.

*Lembre-se de realizar o **git push** com sua tentativa. Não buscamos o código perfeito, mas o registro da sua investigação sobre a representação computacional!*

4 Encontro IV: Deploy e Avaliação (03/06)

Objetivos

Este encontro final é dedicado à publicação e socialização das ferramentas produzidas. O processo de *deploy* (colocação em produção) é o que transforma um exercício de programação em um Recurso Educacional Aberto (REA) disponível para a comunidade docente.

4.1 Configuração de Dependências

Para que o ambiente de nuvem (*Streamlit Cloud*) seja capaz de reproduzir o ecossistema de desenvolvimento utilizado localmente, é imperativo a criação de um arquivo de configuração de dependências. Este arquivo atua como um manifesto, instruindo o servidor sobre quais bibliotecas devem ser instaladas para a execução correta do Mathlet.

Conteúdo e Estrutura

O arquivo deve ser nomeado estritamente como `requirements.txt` (em letras minúsculas) e salvo na raiz do repositório. O conteúdo deve listar as bibliotecas fundamentais utilizadas ao longo dos quatro encontros da oficina:

Conteúdo do requirements.txt

```
streamlit
matplotlib
numpy
```

4.2 Justificativa Teórico-Técnica das Bibliotecas

A escolha destas bibliotecas no processo se justifica pelas seguintes funcionalidades:

- **streamlit:** Provê a infraestrutura para a interface de usuário (UI), permitindo que parâmetros matemáticos abstratos (como ϵ e δ) sejam manipulados via *sliders*.
- **matplotlib:** Biblioteca de visualização de dados responsável pela renderização gráfica dos objetos da Análise, como vizinhanças, funções e partições de Riemann.
- **numpy:** Essencial para o rigor numérico. Permite a criação de partições densas do domínio através do método `linspace` e o tratamento de funções como vetores, garantindo a performance necessária para simulações em tempo real.

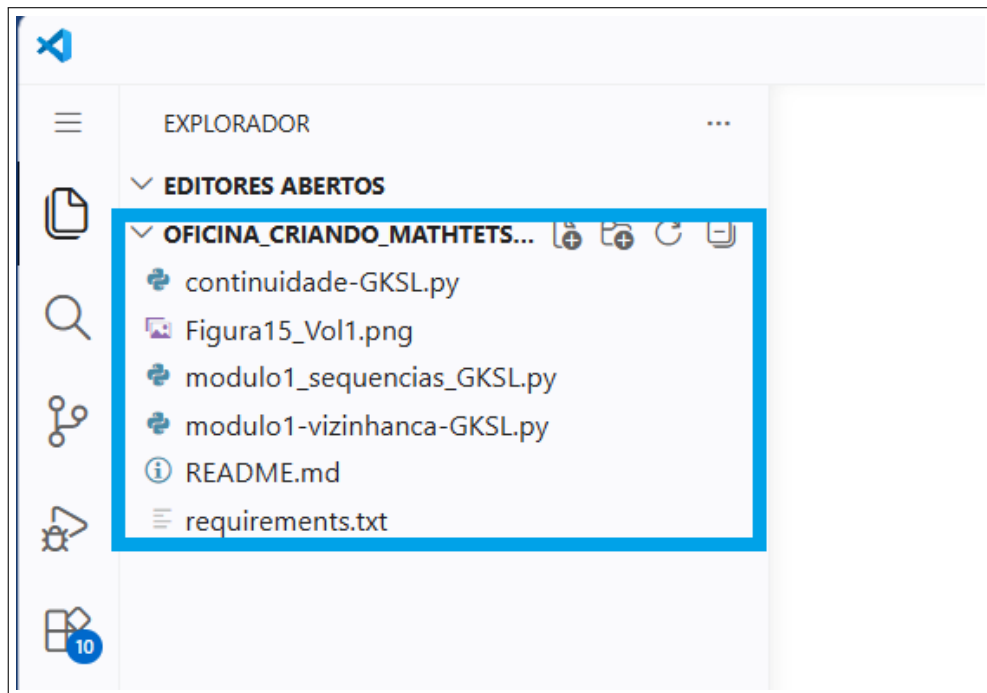
4.3 Passo a Passo para Publicação no Streamlit Cloud

O *deploy* é a etapa final da Gênese Instrumental, onde o seu código deixa de ser um script local e passa a ser uma aplicação web acessível por qualquer navegador através de uma URL pública.

Etapa 1: Preparação do Repositório (GitHub)

Antes de conectar à nuvem, certifique-se de que o seu repositório no GitHub contém os dois arquivos essenciais:

- **O Código principal:** Seus arquivos `.py`.
- **As Dependências:** O arquivo `requirements.txt` configurado corretamente.



Etapa 2: Acesso à Plataforma Streamlit Cloud

1. Acesse [Streamlit Cloud](#).

2. Clique no botão **Continue with GitHub**.

Entrar no Streamlit Community Cloud

 Continuar com Google

 Continuar com GitHub

OU

E-mail

Continuar

Não tem uma conta? [Cadastrar-se](#)

3. Autorize o Streamlit a acessar seus repositórios públicos (isso é necessário para que ele consiga "ler" seu código e publicá-lo).

[← Back](#)

What would you like to do?

**Deploy a public app from GitHub**

My code is ready on a GitHub repo, and it is totally awesome.

[Deploy now](#)

**Deploy a public app from a template**

I want to see what kind of amazing concoctions you have for me.

[Check out templates](#)

**Deploy a private app in Snowflake**

I want unlimited enterprise-grade apps, with the security of Snowflake.

[Start trial →](#)

Etapa 3: Configuração do Deploy

Uma vez logado, siga estes passos:

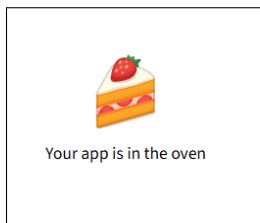
1. Clique no botão azul **Create app** (ou **New app**).



2. No campo **Repository**, selecione o repositório da sua oficina.
3. No campo **Main file path**, selecione o arquivo principal que você deseja que abra primeiro.
4. Em **App URL**, você pode personalizar o nome do link.
5. Clique em **Deploy!**.

A screenshot of the 'Deploy an app' form in Streamlit. The form has a 'Back' link at the top left. The title 'Deploy an app' is prominently displayed. There are four main input fields: 'Repository' (containing 'Greice-Lacerda/Oficina_Criando_Mathtets_Streamlit'), 'Branch' (containing 'main'), 'Main file path' (containing 'modulo1_sequencias_GKSL.py'), and 'App URL (optional)' (containing 'oficinacriandomathtetsapp-nfwqbhaxsqi2ffe8eevpcd'). A 'Paste GitHub URL' link is next to the Repository field. Below the App URL field, a green message states 'Domain is available'. There is a link for 'Advanced settings' and a blue 'Deploy' button at the bottom, which is highlighted with a red box.

Etapa 4: O Processo de “Baking” (Cozimento)

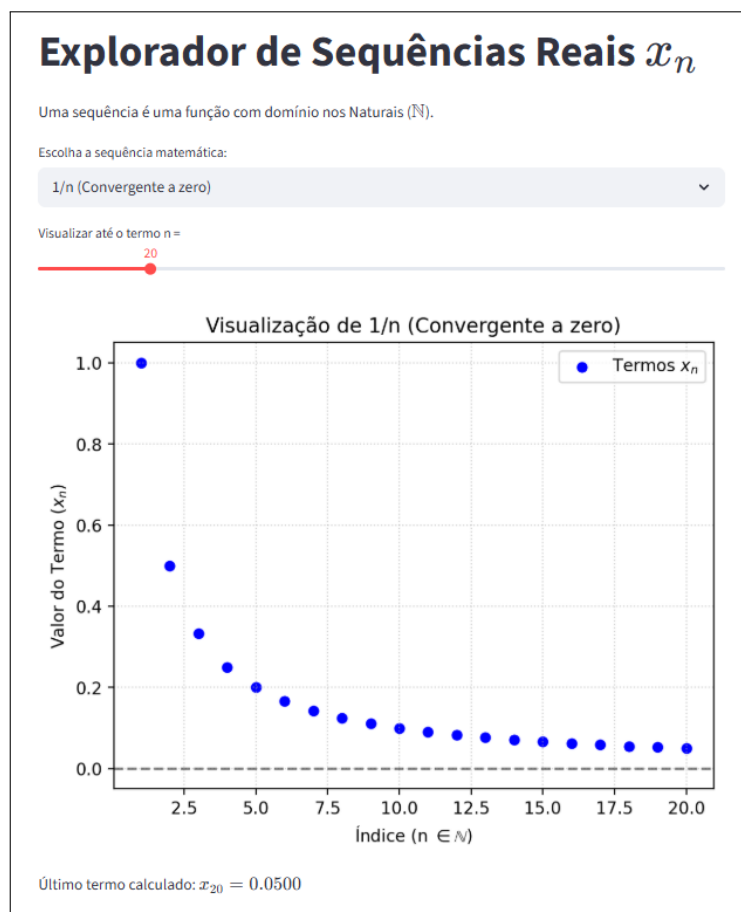


O Streamlit levará alguns minutos para configurar o servidor. Você verá um terminal à direita mostrando a instalação das bibliotecas listadas no seu `requirements.txt`.

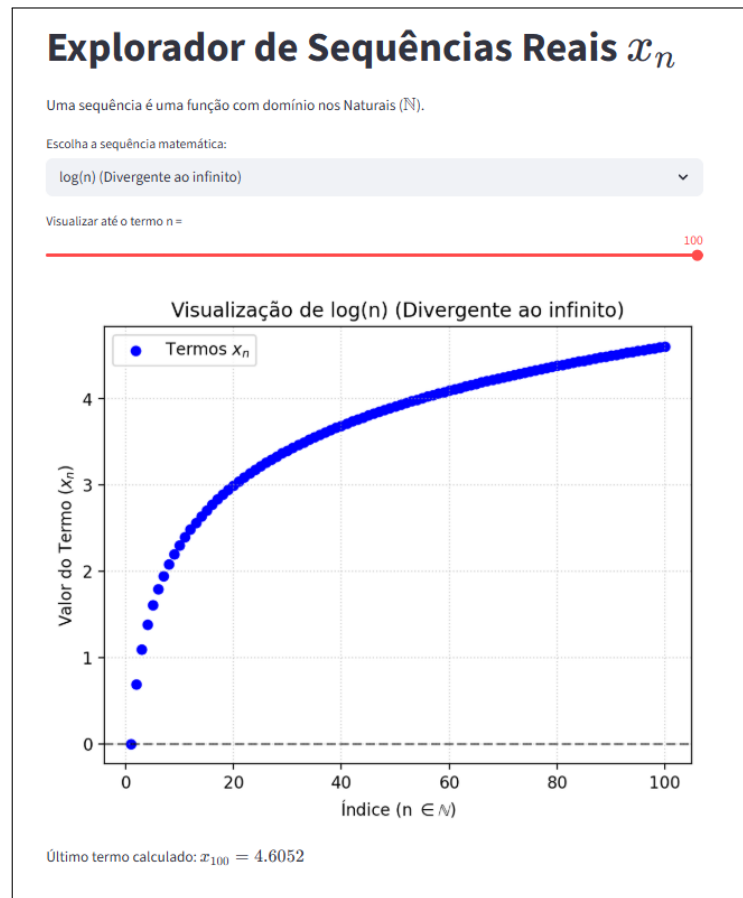
- **Se aparecer um erro:** Verifique se o nome do arquivo no *Main file path* está escrito exatamente igual ao que está no GitHub.
- **Se aparecer sucesso:** Seu Mathlet aparecerá na tela e o link já poderá ser compartilhado.

Veja os dois exemplos construídos nos nossos encontros:

1. [Explorador de Sequências Reais \$x_n\$](#) .



2. Visualizador de Vizinhança $V_\delta(a)$.



4.4 O App Publicado

Ao final do processo, a interface deixa de ser apenas uma janela no Codespaces e torna-se uma página web responsiva. Uma das grandes vantagens da integração GitHub-Streamlit é a **sincronização automática**:

Sempre que você fizer um novo *Push* no GitHub Codespaces, o Streamlit Cloud detectará a mudança e atualizará o site automaticamente em poucos segundos.

Dica Pedagógica: Utilize este link no seu portfólio acadêmico ou envie para seus colegas testarem a interatividade da sua ferramenta.

4.5 Questões Reflexivas: O Professor como Designer

1. **Da Ferramenta ao Pensamento:** Agora que seu aplicativo está online, como você descreve a diferença entre "usar um software pronto" e "construir sua própria ferramenta de ensino"?
2. **A Escolha dos Parâmetros:** Ao projetar os *sliders* de Epsilon ou de Partições, você tomou decisões pedagógicas. Como essas decisões refletem a sua compreensão sobre os obstáculos de aprendizagem dos seus futuros alunos?
3. **Apropriação Tecnológica:** O fato de o código estar aberto no GitHub permite que outros professores adaptem seu material. Como você enxerga essa cultura de compartilhamento na formação do professor de matemática?

Considerações Finais: O Professor como Autor

Ao final desta oficina, esperamos que a percepção sobre a Análise Real tenha transitado de um campo de abstração estática para um território de experimentação dinâmica. A construção dos *Mathlets* e a manipulação de códigos em Python não foram apenas exercícios técnicos, mas sim um esforço de **tradução semiótica** e **ressignificação pedagógica**.

A experiência de autoria aqui vivenciada demonstra que a tecnologia, quando mediada pela intenção docente, não simplifica o rigor matemático, mas o torna visível e manipulável. Ao enfrentar os erros de sintaxe e os desafios de algoritmização, você mobilizou conhecimentos profundos (CK) e pedagógicos (PK), consolidando a base do modelo **TPACK** necessária para a docência na era digital.

Desejamos que as ferramentas exploradas — Streamlit, GitHub Codespaces e o uso assistido de Inteligência Artificial — continuem a servir como "andaimes" para suas futuras práticas. Que este seja apenas o início de uma trajetória onde você, como professor-designer, sinta-se empoderado para criar suas próprias pontes entre o rigor da matemática acadêmica e a realidade da sala de aula.

“A melhor maneira de prever o futuro da educação é criá-lo”

Profa. Dra. Greice Keli Silva Lacerda
Profa. Dra. Márcia Maria Fusaro Pinto
Projeto Math<<Fusion>>Tech / DEPEXT

Referências Bibliográficas

- ABBOTT, Stephen. **Understanding Analysis**. 2. ed. New York: Springer, 2015.
- ELAYDI, Saber. **An Introduction to Difference Equations**. 3. ed. New York: Springer, 2005.
- FIGUEIREDO, Djairo Guedes de. **Análise Real**. Rio de Janeiro: IMPA, 2014.
- LIMA, Elon Lages. **Análise Real: Volume 1**. Rio de Janeiro: IMPA, 2017.
- BORBA, Marcelo de Carvalho; PENTEADO, Miriam Godoy. **Informática e Educação Matemática**. 5. ed. Belo Horizonte: Autêntica, 2012.
- DUVAL, Raymond. **Semiosis e pensamento humano: registros de representação semiótica para o aprendizado intelectual**. São Paulo: Livraria da Física, 2011.
- KOPP, S. G.; JUCHEM-DEL PONT, M. Uma introdução ao conceito de limite através de atividades no GeoGebra. **Revista Eletrônica de Matemática**, v. 4, n. 1, p. 118-135, 2018.
- RABARDEL, Pierre. **Les hommes et les technologies: une approche cognitive des instruments contemporains**. Paris: Armand Colin, 1995.
- TROUCHE, Luc. Managing the complexity of human/machine interactions in computerized learning environments: Guiding students' process of instrumental genesis. **International Journal of Computers for Mathematical Learning**, v. 9, n. 3, p. 281-307, 2004.
- HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, n. 7825, p. 357-362, 2020.
- HUNTER, J. D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90-95, 2007.
- STREAMLIT. **Streamlit Documentation**: Build and share data apps. Disponível em: <https://docs.streamlit.io>. Acesso em: 15 abr. 2026.
- VAN ROSSUM, Guido; DRAKE, Fred L. **Python 3 Reference Manual**. Scotts Valley: CreateSpace, 2009.
- VALENTE, José Armando. **O computador na sociedade do conhecimento**. Campinas: UNICAMP/NIED, 1999.

Apêndice A: Guia de Configuração do Ambiente de Desenvolvimento

Este guia descreve os procedimentos necessários para a preparação do ecossistema tecnológico utilizado na oficina, integrando o *GitHub Codespaces* e as bibliotecas da linguagem *Python*.

1. Acesso ao Repositório e Criação do Codespace

1. Acesse o repositório da oficina no GitHub através do link fornecido pela pesquisadora.
2. Clique no botão verde **Code**, selecione a aba **Codespaces** e clique em **Create codespace on main**.
3. Aguarde a construção do contêiner virtual, que disponibilizará uma interface do *VS Code* diretamente no seu navegador.

2. Instalação das Bibliotecas Necessárias

No terminal do Codespaces, execute o seguinte comando para garantir que todas as ferramentas matemáticas e de interface estejam disponíveis:

```
pip install streamlit matplotlib numpy
```

3. Comandos Essenciais do Terminal

Para a condução das atividades, o licenciando deve estar familiarizado com os seguintes comandos de execução e controle de versão:

Comando	Função
<code>streamlit run arquivo.py</code>	Executa o Mathlet interativo no navegador.
<code>git add .</code>	Prepara as alterações para o salvamento.
<code>git commit -m "mensagem"</code>	Registra a versão atual com uma descrição.
<code>git push origin main</code>	Envia o trabalho para a nuvem do GitHub.

Tabela 1: Principais comandos utilizados durante a oficina.

Este guia visa reduzir a carga cognitiva técnica, permitindo que o foco do licenciando permaneça na exploração dos conceitos de Análise Real.

Apêndice B: Código-Fonte do Visualizador de Vizinhanças

Este apêndice contém o script completo utilizado para a construção do *Mathlet* interativo sobre Vizinhança de um Ponto em $\mathbb{R}$.

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Interface de usuario
6 st.title("Visualizador de Vizinhança  $V_{\{\delta\}}(a)$ ")
7
8 # 1. Parametros da Definicao (Widgets)
9 # 0 slider permite a exploracao dinamica dos valores de centro e raio
10 a = st.slider("Centro (a):", -5.0, 5.0, 0.0)
11 delta = st.slider("Raio ( $\delta$ ):", 0.1, 2.0, 0.5)
12
13 # 2. Construcao do Grafico
14 fig, ax = plt.subplots(figsize=(10, 2))
15 ax.axhline(0, color='black', linewidth=1) # Representacao da Reta Real
16
17 # 3. Traducao do Rigor para o Grafico
18 # Destacamos a area que satisfaz a condicao  $|x - a| < \delta$ 
19 ax.axvspan(a - delta, a + delta, color='green', alpha=0.3,
20           label=f"Vizinhança  $V_{\{\delta\}}(\{a\})$ ")
21
22 # 4. Marcacao do Ponto Central
23 ax.scatter([a], [0], color='red', zorder=5, label="Centro (a)")
24
25 # Ajustes de visualizacao para foco na topologia da reta
26 ax.set_xlim(-10, 10)
27 ax.set_yticks([]) # Remocao do eixo Y para analise unidimensional
28 ax.legend()
29
30 st.pyplot(fig)
31
32 # 5. Explicitacao da Condicao Matematica (Utilizando raw strings
33 para LaTeX)
34 st.latex(rf"x \in \mathbb{R} : |x - \{a\}| < \{\delta\}")
```

Código 3: Script da Vizinhança Dinâmica

Nota: Este código deve ser executado em ambiente com as bibliotecas `streamlit`, `matplotlib` e `numpy` instaladas.

Apêndice C: Protótipo de Mathlet para Convergência de Sequências

Este apêndice apresenta o código-fonte base desenvolvido no Módulo I, utilizando Python e a biblioteca Streamlit para a visualização dinâmica de sequências reais.

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 st.title("Visualizador de Sequências Reais")
6
7 # Entrada do usuário para a definição da sequência
8 formula = st.sidebar.text_input("Defina a sequência a_n
9 (em termos de n):", "1/n")
10 n_max = st.sidebar.slider("Quantidade de termos (n):", 1, 1000, 100)
11
12 def calcular_sequencia(f, n_val):
13     # Uso de eval para processar a string da fórmula com segurança
14     # controlada
15     n = np.arange(1, n_val + 1)
16     return eval(f)
17
18 try:
19     y = calcular_sequencia(formula, n_max)
20     x = np.arange(1, n_max + 1)
21
22     fig, ax = plt.subplots()
23     ax.scatter(x, y, s=10, color="blue")
24     ax.set_xlabel("n")
25     ax.set_ylabel("a_n")
26     ax.grid(True)
27
28     st.pyplot(fig)
29
30     st.write(f"O último termo calculado foi a_{n_max} = {y[-1]}")
31
32 except Exception as e:
33     st.error(f"Erro na fórmula matemática: {e}")
```

Código 4: Script principal do Mathlet de Sequências

Apêndice D: Estrutura de Integração (Somas de Riemann)

Código referente ao Módulo III, focando no particionamento do intervalo $[a, b]$ e no cálculo das somas superiores e inferiores.

```
1 # Fragmento do cálculo da área sob a curva
2 import streamlit as st
3 import numpy as np
4
5 st.header("Exploração de Somas de Riemann")
6
7 a = st.number_input("Limite Inferior (a):", value=0.0)
8 b = st.number_input("Limite Superior (b):", value=1.0)
9 n = st.slider("Número de partições:", 2, 100, 10)
10
11 dx = (b - a) / n
12 x_part = np.linspace(a, b, n+1)
13
14 # Exemplo de função f(x) = x^2
15 def f(x):
16     return x**2
17
18 soma_esquerda = sum(f(x_part[:-1]) * dx)
19 st.metric("Soma de Riemann (Esquerda)", f"{soma_esquerda:.4f}")
```

Código 5: Algoritmo de Somas de Riemann

Apêndice E: Código-Fonte para Análise de Continuidade Local ($\epsilon - \delta$)

Este apêndice apresenta o script desenvolvido para a exploração dinâmica da definição de continuidade em um ponto. O código materializa a dependência entre as vizinhanças do domínio (δ) e da imagem (ϵ), transformando a verificação formal em um processo algorítmico de busca.

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Interface de Usuario
6 st.title("Laboratorio de Analise: Investigando a Continuidade Local")
7
8 # --- 1. Definicao do Banco de Funcoes ---
9 # Criamos funcoes Python para simular cenarios analiticos
10 def funcao_continua_polinomial(x):
11     return x**2
12
13 def funcao_continua_seno(x):
14     return np.sin(x)
15
16 def funcao_descontinua_salto(x):
17     # Funcao degrau: 0 se x < 0, 1 se x >= 0
18     return np.where(x < 0, 0, 1)
19
20 def funcao_descontinua_removivel(x):
21     # Funcao com 'buraco' em x=0: (x^2)/x, mas x!=0
22     # Usamos np.nan para representar a indeterminacao
23     y = x.copy()
24     y[np.abs(x) < 0.01] = np.nan # Cria o 'buraco' proximo a zero
25     return y
26
27 # Dicionario para organizar a selecao na interface
28 banco_funcoes = {
29     "Polinomial Cont nua ( $x^2$ )": funcao_continua_polinomial,
30     "Trigonometrica Cont nua ( $\sin(x)$ )": funcao_continua_seno,
31     "Descontinuidade de Salto (Degrau)": funcao_descontinua_salto,
32     "Descontinuidade Removivel ('Buraco')": funcao_descontinua_removivel
33 }
34
35
36 # --- 2. Interface de Selecao e Parametros (Sidebar) ---
37 st.sidebar.header("Configuracao da Analise")
38
```

```

39 # Selecao da Funcao
40 nome_funcao = st.sidebar.selectbox("Selecione a Funcao:",
41 list(banco_funcoes.keys()))
42 funcao_escolhida = banco_funcoes[nome_funcao]
43
44 # Parametros Analiticos
45 a = st.sidebar.slider("Ponto de Analise (a):", -2.0, 2.0, 1.0)
46 epsilon = st.sidebar.slider("Erro na Imagem (Epsilon):", 0.05, 1.0, 0.2)
47 delta = st.sidebar.slider("Margem no Dominio (Delta):", 0.01, 1.0, 0.1)
48
49 # --- 3. Processamento Analitico ---
50 # Amostragem densa para garantir rigor visual
51 x = np.linspace(a - 3, a + 3, 1000)
52 y = funcao_escolhida(x)
53
54 # Calculo de f(a) - Cuidado com indeterminacoes
55 try:
56     f_a = funcao_escolhida(np.array([a]))[0]
57 except:
58     f_a = np.nan # Se f(a) nao existir (ex: removivel)
59
60 # --- 4. Construcão do Grafico (Mantendo a Logica Anterior) ---
61 fig, ax = plt.subplots(figsize=(8, 6))
62 ax.plot(x, y, color='black', label=f'f(x)')
63
64 # Se f(a) existe, plotamos as vizinhanças e o ponto
65 if not np.isnan(f_a):
66     # Faixa de Epsilon (Imagem)
67     ax.axhspan(f_a - epsilon, f_a + epsilon, color='blue', alpha=0.15,
68 label=rf'Vizinhança a  $V_{\epsilon}(f(a))$ ')
69     # Faixa de Delta (Dominio)
70     ax.axvspan(a - delta, a + delta, color='green', alpha=0.15,
71 label=rf'Vizinhança a  $V_{\delta}(a)$ ')
72
73     # Marcacao do Ponto (a, f(a))
74     ax.scatter([a], [f_a], color='red', zorder=5)
75 else:
76     # Se f(a) nao existe, plotamos apenas a vizinhança delta e um aviso
77     ax.axvspan(a - delta, a + delta, color='green', alpha=0.15,
78 label=rf'Vizinhança a  $V_{\delta}(a)$ ')
79     st.warning(f"Atenção: A função não está definida no ponto a={a}.")
80
81
82 # Rigor Visual: Eixos e Grid
83 ax.axhline(0, color='black', lw=0.8)
84 ax.axvline(0, color='black', lw=0.8)
85 ax.set_xlabel("Dominio (x)")

```



```

86 ax.set_ylabel("Imagem (f(x))")
87 ax.legend()
88 ax.set_title(f"Análise de Continuidade: {nome_funcao}")
89 st.pyplot(fig)
90
91 # --- 5. Verificacao da Condicao Analitica (Mantendo a Logica) ---
92 st.markdown("---")
93 st.latex(rf"|x - {a}| < {delta} \implies |f(x) - f({a})| < {epsilon}?")
94
95 if not np.isnan(f_a):
96     # Testamos se as extremidades da vizinhanca delta permanecem dentro
97     #de epsilon
98     # Nota: Para rigor maximo, deveria testar *todos* os pontos da
99     #amostragem no delta
100    imagem_no_delta = np.array([funcao_escolhida(np.array([a - delta]))
101                                [0],
102                                funcao_escolhida(np.array([a + delta]))
103                                [0]])
104    erro_max = np.max(np.abs(imagem_no_delta - f_a))
105
106    if erro_max <= epsilon:
107        st.success(f"Condicao Satisfeita! $\delta={delta}$ suficiente
108        para $\epsilon={epsilon}$ em $a={a}$.")
109    else:
110        st.error(f"Condicao Violada! Para $\epsilon={epsilon}$,
111        o $\delta={delta}$ eh muito grande.")
112 else:
113     st.error("A condicao de continuidade falha pois f(a) nao esta
114     definida.")

```

Código 6: Explorador Comparativo de Continuidade ($\epsilon - \delta$)

Nota: Este código exemplifica a conversão da linguagem matemática para a lógica booleana, permitindo a validação experimental de teoremas de Análise Real.

Apêndice F: Código-Fonte para Somas de Riemann e Cálculo de Erro

Este apêndice apresenta o script desenvolvido para o Módulo III, focado na visualização da integral definida. O código inclui a lógica de partição do intervalo $[a, b]$ e a comparação algorítmica entre a aproximação numérica (Soma de Riemann) e o valor exato obtido pelo Teorema Fundamental do Cálculo.

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 st.title("Somas de Riemann: Do Discreto ao Contínuo")
6
7 # 1. Entrada de Parametros (Interface Lateral)
8 st.sidebar.header("Parâmetros da Integral")
9 a = st.sidebar.number_input("Início (a):", value=0.0)
10 b = st.sidebar.number_input("Fim (b):", value=2.0)
11 n = st.sidebar.slider("Número de Partições (n):", 1, 200, 10)
12
13 # 2. Definição da Função e sua Integral Analítica
14 def f(x):
15     return x**2
16
17 def integral_exata(a, b):
18     # Integral de x^2 e x^3/3
19     return (b**3 / 3) - (a**3 / 3)
20
21 # 3. Cálculo da Soma de Riemann (Extremidade Esquerda)
22 dx = (b - a) / n
23 pontos_x = np.linspace(a, b - dx, n)
24 soma_riemann = np.sum(f(pontos_x) * dx)
25
26 # 4. Cálculo do Erro
27 valor_real = integral_exata(a, b)
28 erro_absoluto = abs(valor_real - soma_riemann)
29
30 # 5. Visualização Gráfica
31 fig, ax = plt.subplots(figsize=(8, 5))
32 x_curva = np.linspace(a, b, 200)
33 ax.plot(x_curva, f(x_curva), color='red', label=r'$f(x) = x^2$')
34
35 # Desenho dos retângulos da partição
36 for i in range(n):
37     xi = a + i*dx
```

```

38     ax.add_patch(plt.Rectangle((xi, 0), dx, f(xi),
39                             edgecolor='blue', facecolor='blue', alpha=0.2))
40
41 ax.set_title(f"Soma de Riemann com n={n}")
42 ax.set_xlabel("x")
43 ax.set_ylabel("f(x)")
44 ax.legend()
45 st.pyplot(fig)
46
47 # 6. Exibicao dos Resultados Analiticos
48 col1, col2 = st.columns(2)
49 with col1:
50     st.metric("Soma de Riemann", f"{soma_riemann:.5f}")
51     st.metric("Integral Exata", f"{valor_real:.5f}")
52
53 with col2:
54     st.metric("Erro Absoluto", f"{erro_absoluto:.5f}")
55
56 st.markdown("---")
57 st.latex(rf"\text{{Erro}} = \left| \int_{{a}}^{{b}} x^2 \right. \\
58 dx - \sum_{{i=1}}^{{n}} f(x_i) \Delta x \left. \right| = \\
59 {erro_absoluto:.5e}")
60
61 if erro_absoluto < 0.01:
62     st.success("Precis o atingida! O erro     menor que 0.01.")
63 else:
64     st.warning("Aumente o valor de 'n' para melhorar a aproxima o.")

```

Código 7: Somas de Riemann com Cálculo de Erro Absoluto

Nota: A utilização da notação científica (e) no erro absoluto permite observar a convergência da sequência de somas para o valor real da integral.

Apêndice G: Instrumentos de Coleta de Dados (Questionários de Encontro)

Como parte da metodologia desta pesquisa, ao final de cada encontro da oficina, os participantes responderam a um formulário eletrônico (Google Forms) contendo questões estruturadas para capturar o processo de Gênese Instrumental e o desenvolvimento da Autoria Docente.

- **Dimensão Técnica:** Avalia a superação de obstáculos com a sintaxe do Python e uso de IA. [Lista de presença para o dia 22/04/2026](#)
- **Dimensão Cognitiva:** Foca na transição entre os registros de representação (Gráfico, Algébrico, Algorítmico). [Lista de presença para o dia 06/05/2026](#).
- **Dimensão Profissional:** Investiga a percepção do licenciando sobre sua futura atuação como produtor de tecnologias. [Lista de presença para o dia 22/05/2026](#).

As respostas coletadas compõem o corpus de análise qualitativa desta tese, permitindo o mapeamento da evolução de cada sujeito ao longo das etapas de implementação e deploy dos *Mathlets*. [Lista de presença para o dia 03/06/2026](#).